

# Payment Gateway: Design and Implementation

Oct 6, 2026

## 1. Overview

A merchant sends us card details and an amount. The gateway checks them, asks the acquiring bank for approval, safely stores the result and lets the merchant look it up later.

### Scope delivered

- Process a payment with POST /api/payments, returning Authorized, Declined or Rejected.
- Retrieve a payment with GET /api/payments/{id}.
- Every field validated before the bank is called.
- Bank integration through a dedicated adapter, including outages and timeouts.
- 46 automated tests across four layers, run in CI on every push.
- Health check at GET /health, structured logging and Docker packaging.

**Principle:** keep it simple, and get the payment details right: card data handling, honest failures and clear contracts.

**Stack:** C#, .NET 8, ASP.NET Core, xUnit, Docker, GitHub Actions.

### Assessed areas

| Area                   | How it is covered                                                                                                                            | Section |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|---------|
| <b>API Design</b>      | A status code per outcome, Location header on create, one Rejected shape, status as text, Swagger                                            | 7       |
| <b>Code Design</b>     | One job per class, bank behind an adapter, storage behind a repository, result type instead of exceptions, no field that can store card data | 3, 4, 6 |
| <b>Testing Mindset</b> | Baseline run first, 46 tests in four layers, boundary values, fakes for the clock, network and bank, CI                                      | 8.1     |
| <b>Observability</b>   | Structured logs without card data, warnings on bank failures, health endpoint, trace id on errors                                            | 8.2     |

| Area                           | How it is covered                                                                                | Section |
|--------------------------------|--------------------------------------------------------------------------------------------------|---------|
| <b>Packaging/Hosting</b>       | Configuration per environment, multi-stage non-root Docker image, compose with the simulator, CI | 8.3     |
| <b>Technical Documentation</b> | This document, the README, Swagger and small single-purpose commits                              | All     |

## 2. Requirements

Three contracts define the system.

### Merchant to gateway (all fields required)

| Field        | Rule                                          |
|--------------|-----------------------------------------------|
| Card number  | 14 to 19 characters, numeric only             |
| Expiry month | 1 to 12                                       |
| Expiry year  | Month and year together must be in the future |
| Currency     | 3-letter ISO code, one of GBP, USD, EUR       |
| Amount       | Integer in minor units (£10.50 = 1050)        |
| CVV          | 3 to 4 characters, numeric only               |

### Gateway to merchant

| Field                 | Note                              |
|-----------------------|-----------------------------------|
| Id                    | Unique GUID                       |
| Status                | Authorized, Declined or Rejected  |
| Last four card digits | The full number is never returned |
| Expiry month and year | As submitted                      |
| Currency              | As submitted                      |
| Amount                | Minor units                       |

### Bank simulator

| Condition                   | Response                                   |
|-----------------------------|--------------------------------------------|
| A required field is missing | 400 Bad Request                            |
| Card ends in an odd digit   | 200, authorized with an authorization code |
| Card ends in an even digit  | 200, not authorized                        |
| Card ends in 0              | 503 Service Unavailable                    |

The bank decides Authorized or Declined. The gateway decides Rejected, before the bank is ever called.

### 3. Design options considered

I chose **B, simple layers in one project**. Each part has one job and can be tested alone, without the ceremony the brief warns against.

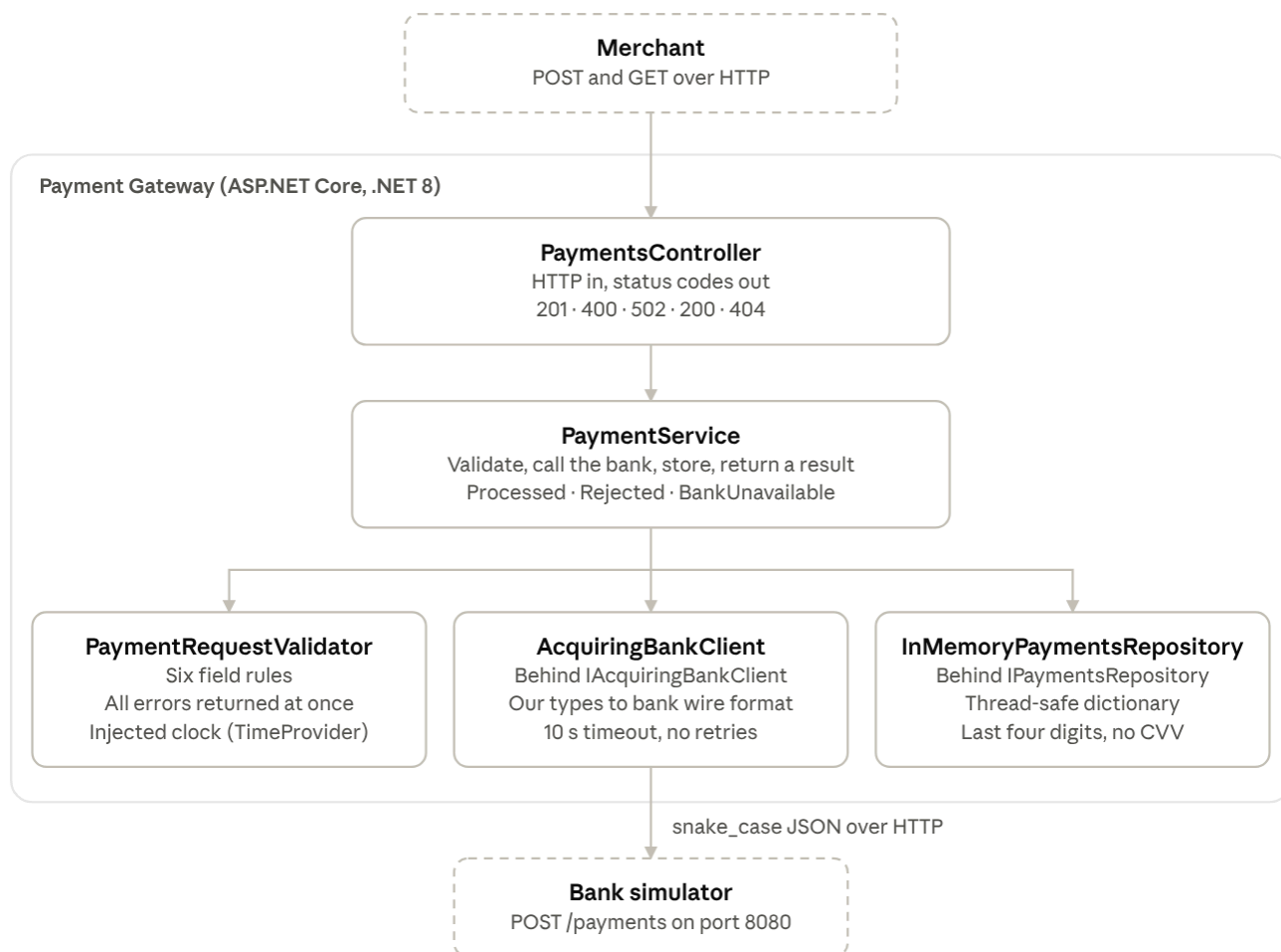
| Option                                   | How it works                                                    | Pros                                                               | Cons                                                                                              |
|------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>A. Everything in the controller</b>   | One class does HTTP, validation, the bank call and storage      | Fastest to write                                                   | Rules can't be tested without HTTP. Bank details leak into the API. One edit can break everything |
| <b>B. Simple layers (chosen)</b>         | Controller, service, validator, bank adapter, repository        | Each part testable alone. Readable. A bank change touches one file | More files                                                                                        |
| <b>C. Clean Architecture as projects</b> | Separate API, Application, Domain and Infrastructure projects   | The compiler enforces who can use whom                             | Over-engineered for two endpoints. More files and mapping                                         |
| <b>D. Queue-based</b>                    | API saves the payment as Pending, a worker calls the bank later | Absorbs slow banks and traffic spikes at scale                     | The brief needs an instant result. Adds a queue, a worker and polling                             |

**Why not C:** B already uses C's key idea, interfaces at the bank and storage boundaries. Separate projects pay off when several teams share the code.

**Why not D:** it suits production scale (see 9.3), but the merchant here needs an immediate answer.

## 4. Architecture

Each request passes through four layers, and only the adapter talks to the bank



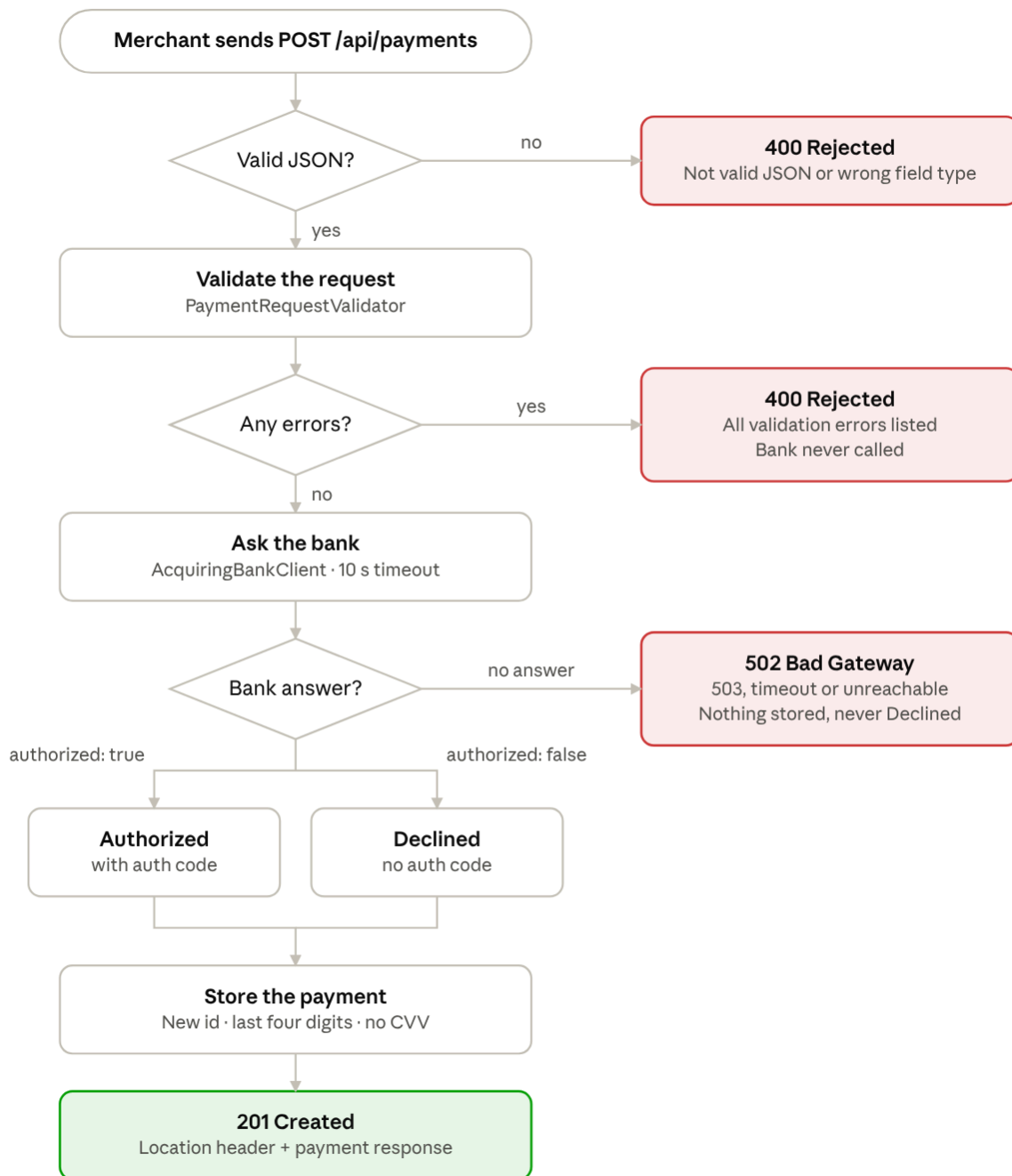
Dashed boxes sit outside the gateway.

Architecture · controller, service, three dependencies, bank

The controller translates HTTP, the service runs the steps, and each dependency has one job. The bank and storage sit behind interfaces, so tests use fakes and production can swap in real infrastructure.

## 5. Payment flow

Only a valid request that gets a bank answer creates a payment



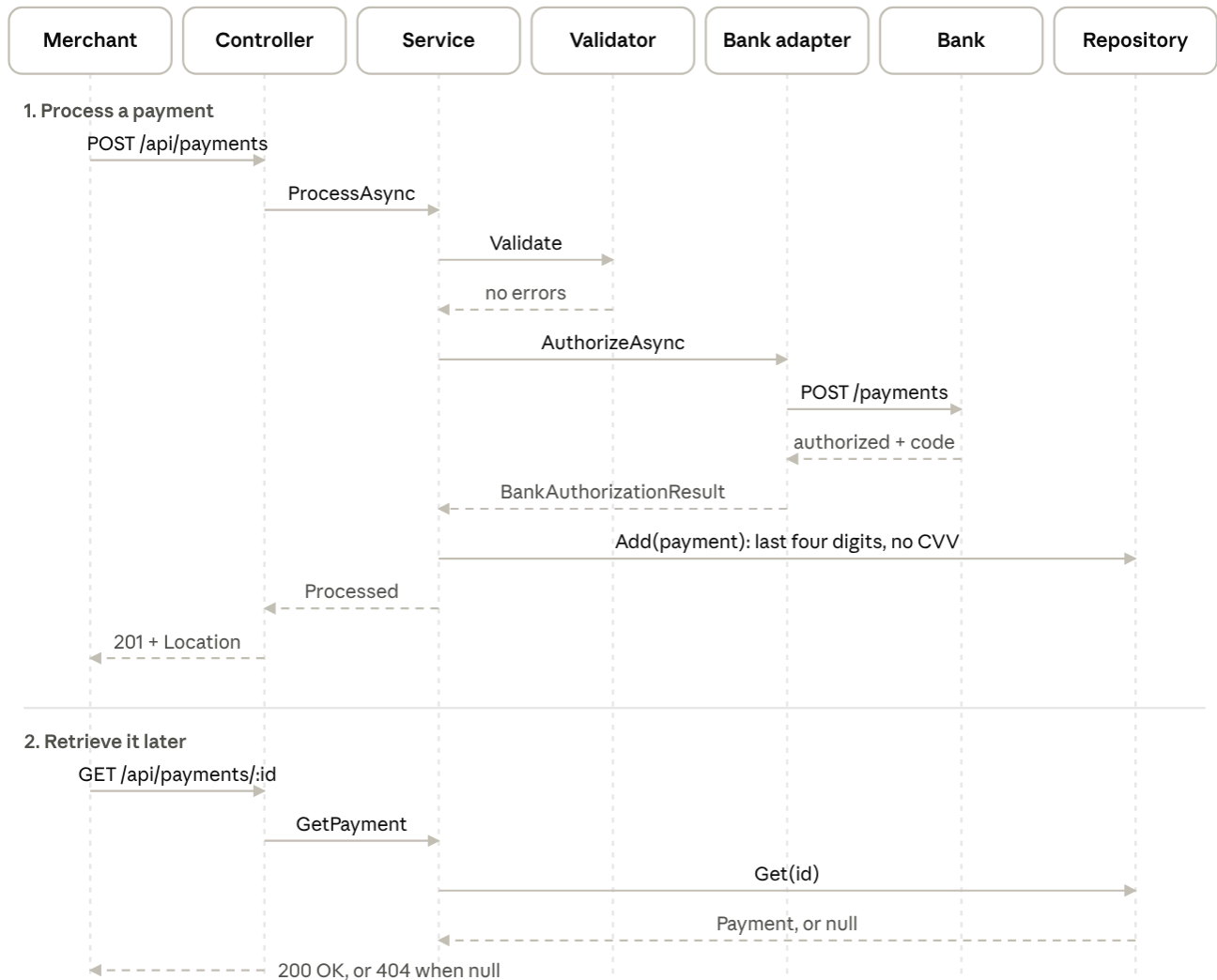
Red: refused or failed, nothing stored. Green: payment created.

POST /api/payments · 3 decisions, 4 outcomes

The gateway decides Rejected before the bank is called. The bank decides Authorized or Declined. No answer from the bank means 502 and nothing stored, because the outcome is unknown, not a decline.

## User journey: who calls whom

A payment calls the bank once, and a lookup never does



User journey · who calls whom, process then retrieve

Solid arrows are calls, dashed arrows are returns. The other journeys leave this path at fixed points.

| Journey                    | Leaves at                     | Stored                       | Merchant gets   |
|----------------------------|-------------------------------|------------------------------|-----------------|
| Authorized (card ends odd) | Completes every step          | Yes, with authorization code | 201, Authorized |
| Declined (card ends even)  | Bank says not authorized      | Yes, no authorization code   | 201, Declined   |
| Bank down (card ends 0)    | Bank returns 503 or times out | No                           | 502 Bad Gateway |

| Journey              | Leaves at                    | Stored         | Merchant gets             |
|----------------------|------------------------------|----------------|---------------------------|
| Invalid fields       | Validator, bank never called | No             | 400, Rejected with errors |
| Malformed JSON       | Before the controller runs   | No             | 400, Rejected             |
| Retrieve, known id   | Repository finds it          | Already stored | 200 OK                    |
| Retrieve, unknown id | Repository finds nothing     | Not applicable | 404 Not Found             |

## 6. Implementation

For each component: the need, what was built, and why.

### 6.1 Models (Models folder)

| Model                   | Role                    | Decision                                                                                                                                                          |
|-------------------------|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PostPaymentRequest      | What the merchant sends | CardNumber and Cvv are strings, so CVV 012 stays 012. Fields are nullable, so a missing field reads "required", not 0. Amount is a long. Properties are init-only |
| Payment (record)        | What we store           | CardNumberLastFour only. No field for the CVV or full card number, so they cannot be stored                                                                       |
| PaymentResponse         | What the merchant sees  | Replaces two identical template classes. PaymentResponse.From(payment) is the one mapping that decides what leaves the gateway                                    |
| RejectedPaymentResponse | A refusal               | Status Rejected plus the errors. No id, since nothing was created                                                                                                 |

Status is sent as text through JsonStringEnumConverter, so merchants read "Authorized" instead of 0.

### 6.2 Validation (Services/PaymentRequestValidator.cs)

- **Need:** reject bad requests before the bank sees them.
- **Built:** PaymentRequestValidator checks all six fields and returns every error at once.

- **Why:** the merchant fixes everything in one go. A card works until the end of its expiry month. `char.IsAsciiDigit` accepts only 0 to 9. An injected `TimeProvider` keeps date tests stable.

### 6.3 Storage (`Services/IPaymentsRepository.cs`, `InMemoryPaymentsRepository.cs`)

- **Need:** the template's public `List` was unsafe under concurrent requests, and `FirstOrDefault` returned null while promising a payment (the 204 bug).
- **Built:** a private `ConcurrentDictionary` behind `IPaymentsRepository`. `Get` returns `Payment?`, and `TryAdd` refuses duplicate ids.
- **Why:** concurrent payments are safe, lookups are instant, unknown ids return 404, and a database can replace it later.

### 6.4 Bank adapter (`Bank/AcquiringBankClient.cs`)

- **Need:** talk to the bank without its format leaking into the code.
- **Built:** `AcquiringBankClient`, behind `IAcquiringBankClient`, translates our types to the bank's snake\_case JSON and back. Registered with `AddHttpClient`, a 10-second timeout and the URL from `AcquiringBank:BaseUrl`.
- **Why:**
  - A bank format change touches one file.
  - A bank outage returns 502, never Declined. A false decline blames the shopper's card.
  - No automatic retry. The bank may already have charged the card.

### 6.5 Orchestration (`Services/PaymentService.cs`)

- **Need:** one place that runs the steps in order.
- **Built:** `PaymentService.ProcessAsync` validates, calls the bank, stores, then returns a `ProcessPaymentResult`: `Processed`, `Rejected` or `BankUnavailable`.
- **Why:** expected outcomes are results, not exceptions. `PaymentService` is scoped, so the bank's `HttpClient` keeps recycling connections.

### 6.6 API (`Controllers/PaymentsController.cs`)

- **Need:** the template had no `POST /api/payments`, and `GET /api/payments/{id}` returned 204 for unknown payments.
- **Built:** a thin controller on `ControllerBase` that maps each result to its status code: 201 with a `Location` header, 400, 502, 200 or 404.

- **Why:** HTTP concerns stay in one place. Malformed JSON also gets the Rejected shape through InvalidModelStateResponseFactory, with a generic message that echoes no card data.

## 6.7 Production readiness

- **Health check:** GET /health tells the load balancer the instance is alive. It skips the bank, so a bank outage never restarts healthy gateways.
- **No HTTPS redirect:** UseHttpsRedirection removed. TLS ends at the load balancer, and a redirect can turn a POST into a GET.
- **CI:** .github/workflows/ci.yml builds and tests on a clean machine on every push.
- **Docker:** the Dockerfile and docker-compose.yml run the API and the bank simulator with one command.

## 7. API reference

Every outcome has its own status code, so a merchant's system can act on it automatically.

| Request                | Outcome                | Response                              |
|------------------------|------------------------|---------------------------------------|
| POST /api/payments     | Authorized or Declined | 201 Created, Location header, payment |
| POST /api/payments     | Invalid fields         | 400, Rejected with errors             |
| POST /api/payments     | Malformed JSON         | 400, Rejected                         |
| POST /api/payments     | Bank unavailable       | 502 Bad Gateway                       |
| GET /api/payments/{id} | Found                  | 200 OK, payment                       |
| GET /api/payments/{id} | Not found              | 404 Not Found                         |
| GET /health            | Instance alive         | 200, Healthy                          |

Declined returns 201 because a payment was still created.

### Example request

```
{
  "cardNumber": "2222405343248877",
  "expiryMonth": 4,
  "expiryYear": 2030,
  "currency": "GBP",
  "amount": 100,
  "cvv": "123"
}
```

### Example response, 201 Created

```
{
  "id": "3f2a9c1e-5b7d-4e8a-9c21-7d4e5f6a8b90",
  "status": "Authorized",
  "cardNumberLastFour": "8877",
  "expiryMonth": 4,
  "expiryYear": 2030,
  "currency": "GBP",
  "amount": 100
}
```

### Example response, 400 Rejected

```
{
  "errors": ["CVV must be 3-4 numeric characters."],
  "status": "Rejected"
}
```

Swagger documents every endpoint at /swagger in Development.

## 8. Testing, observability, packaging

### 8.1 Testing

46 tests across four layers. None need Docker.

| Test class                   | Fake used                             | Proves                                         |
|------------------------------|---------------------------------------|------------------------------------------------|
| PaymentRequestValidatorTests | FixedTimeProvider (fixed clock)       | Every rule at its boundaries                   |
| AcquiringBankClientTests     | StubHttpMessageHandler (fake network) | Correct bank format and all four bank outcomes |

| Test class                                           | Fake used                                 | Proves                                                                                                  |
|------------------------------------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------|
| PaymentServiceTests                                  | FakeBankClient                            | Invalid requests never reach the bank. Only the last four digits are stored. Outages are never Declined |
| ProcessPaymentEndpointTests, PaymentsControllerTests | WebApplicationFactory with FakeBankClient | Real status codes and JSON over HTTP                                                                    |

## Approach

- **Baseline first.** The untouched template already had a failing test (204 instead of 404).
- **One change per test.** A failure points to one field.
- **Boundaries.** Card lengths 13, 14, 19 and 20. Last month, this month and next month.
- **Data that exposes bugs.** CVV 012 proves the leading zero survives.

## 8.2 Observability

- Structured logging with ILogger message templates, e.g. Payment {PaymentId} processed with status {Status}.
- A warning for every bank failure in AcquiringBankClient.
- No card numbers or CVVs in any log.
- Health endpoint at GET /health, and a traceId on ProblemDetails error responses.

## 8.3 Packaging and running

- **CI:** .github/workflows/ci.yml builds and tests every push on GitHub Actions.
- **Configuration:** AcquiringBank:BaseUrl in appsettings.json, overridden per environment by AcquiringBank\_\_BaseUrl.
- **Docker:** a multi-stage Dockerfile running as a non-root user. docker-compose.yml runs it next to bank\_simulator.

```
# API and bank simulator in Docker (API on http://localhost:5080)
docker compose up -d --build

# Or run the API locally against the simulator (Swagger on
http://localhost:5067/swagger)
docker compose up -d bank_simulator
dotnet run --project src/PaymentGateway.Api

# Tests (no Docker needed)
dotnet test
```

## 9. Template changes, assumptions, next steps

### 9.1 Changes to the template

| Found                                                                    | Changed to                                         |
|--------------------------------------------------------------------------|----------------------------------------------------|
| NuGet packages with known vulnerabilities                                | Updated                                            |
| GET returned 204 for an unknown payment                                  | Returns 404                                        |
| Request carried only the last four digits, as a number. CVV was a number | Full card number and CVV as strings                |
| Two identical response classes                                           | One PaymentResponse                                |
| Status sent as a number                                                  | Sent as text                                       |
| Public list, not thread-safe                                             | Private thread-safe repository behind an interface |
| MVC base class and a false async method                                  | API base class, no false async                     |
| HTTPS redirection                                                        | Removed                                            |

### 9.2 Assumptions

- A card is valid until the end of its expiry month.
- Amount is in minor units and above zero.
- Currencies are GBP, USD and EUR, uppercase only.
- Rejected requests are not stored.
- Declined payments are stored without an authorization code.
- Merchant authentication is out of scope.

## 9.3 Next steps

1. Idempotency keys for safe retries.
2. Merchant authentication, so each merchant sees only its own payments.
3. A database, and card tokenisation to reduce PCI scope.
4. A Pending state and reconciliation for bank timeouts.
5. Metrics and tracing, with alerts on authorization rate and bank latency.
6. Upgrade to .NET 10 before .NET 8 support ends in November 2026.